FastAPI Backend Implementation

Inventory app backend created for the React frontend and PostgreSQL persistence.

What Was Built

I added a separate FastAPI backend under the backend directory. It exposes the asset API contract that the React frontend is already prepared to call through VITE_API_BASE_URL.

- Created a FastAPI application entry point at backend/app/main.py.
- Added CORS configuration so the Vite frontend can call the API during development.
- Added SQLAlchemy database setup with a PostgreSQL-ready DATABASE_URL.
- Added an Asset SQLAlchemy model that stores the current nested inventory shape using JSON columns.
- Added Pydantic schemas matching the frontend asset object shape.
- Added CRUD functions for list, create, update, and delete behavior.
- Added /assets routes for GET, POST, PUT, and DELETE.
- Added backend setup documentation and environment examples.

Backend Files Added

```
backend/requirements.txt
backend/.env.example
backend/README.md
backend/app/main.py
backend/app/config.py
backend/app/database.py
backend/app/models.py
backend/app/schemas.py
backend/app/crud.py
backend/app/routes/assets.py
```

API Endpoints

- GET /health - verifies the backend is running.
- GET /assets - returns all non-deleted asset records.
- POST /assets - creates a new asset and returns the saved asset.
- PUT /assets/{id} - updates an existing asset and returns the saved asset.
- DELETE /assets/{id} - soft-deletes an asset and returns 204 No Content.

Database Design

The first backend version uses one assets table. Core searchable fields such as id, name, department, username, and dateAsOf are stored as normal columns. Nested hardware data is stored as JSON so the backend can accept the same object shape that the frontend already uses.

- cpu, network, peripherals, and system are JSON objects.
- ram, gpu, and storage are JSON arrays because each asset can have multiple entries.
- is_deleted enables soft delete so records can be hidden without immediately removing the database row.
- version increments when records are updated, restored, or deleted.

Frontend Connection

The React app connects to this backend when VITE_API_BASE_URL is configured in the frontend .env file. If that variable is not set, the frontend still uses localStorage for development.

```
VITE_API_BASE_URL=http://localhost:8000
```

Backend Setup Steps

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
uvicorn app.main:app --reload
```

PostgreSQL Configuration

The default backend database URL expects a local PostgreSQL database named invent. Update backend/.env if your username, password, host, port, or database name are different.

```
DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/invent
```

Validation Performed

- Python syntax compilation passed for backend/app.
- The frontend production build passed after wiring create, update, delete, and read requests.

Recommended Next Steps

- Create the PostgreSQL database named invent.
- Install backend dependencies in a virtual environment.
- Run uvicorn app.main:app --reload from the backend directory.
- Set VITE_API_BASE_URL in the frontend .env file and restart Vite.
- Later, add Alembic migrations before production deployment.